2021年11月23日

Python入門

「Python対応 ディジタル信号処理」のための

川又 政征（東北職業能力開発大学校），
阿部 正英（石巻専修大学），八巻 俊輔（東北大学）

「Python 対応 ディジタル信号処理」のための Python 入門

川又 政征（東北職業能力開発大学校），
阿部 正英（石巻専修大学），八巻 俊輔（東北大学）

● 本資料の目的

本資料は森北出版㈱発行の書籍
　樋口監修，阿部，八巻，川又著「Python 対応 ディジタル信号処理」（2021年８月）
（以下，Python 対応 DSP と省略する）
により**ディジタル信号処理を学ぶ人たちに Python の利用法に関しての最小限の知識を与えるために**書かれたものである．具体的には Python 対応 DSP 中に掲載されている Python のプログラムを理解し，実行することができること，また例題のプログラムを修正して新しいプログラムを自作できる程度の知識を与えることを目的としている．このため Python に関しての包括的な知識は与えられていない．

● 本資料の対象者

本資料の対象者は**ディジタル信号処理の学習者であるが，ほんの少しだけのプログラミングの経験があることを想定**している．どのような言語であれ，ウィンドウ環境の PC で，例えば 1 から 10 までの総和や階乗の計算などの簡単なプログラミングの経験があればよい．このためウィンドウ画面の操作やプログラミングの基礎は説明されていない．もし MATLAB, Octave, Scilab のようなスクリプト言語の経験がある人にとっては，これらの言語と Python には多くの類似性があるので，本資料を簡単に眺めるだけで Python に関して最小限の知識が得られるだろう．

● ディジタル信号処理に必要な Python の機能

ディジタル信号処理において必要とされる数学と Python の機能はかなり限定的で単純なものである．数学としては，整数・実数・複素数・ベクトル・行列の計算，三角関数，差分方程式，ほんの少しの複素関数の計算であるが，その深い内容を知っている必要はない．この数学に対応する Python の機能として，**様々な型の数値データ（整数・実数・複素数・ベクトル・行列）に対しての加減乗除と関数（三角関数・指数関数・対数など）の適用，繰り返しと条件分岐などの基本的制御フロー, および２次元と３次元のグラフィックス**が必要である．このため本資料は Python の多様な機能の中からディジタル信号処理に必須の機能だけを取り上げている．本資料では Python の計算機言語としての文法や用語の厳密さにはこだわらずに簡単で定性的な説明が行われていることをご容赦願いたい．

目次

- 本資料の目的 1
- 本資料の対象者 1
- ディジタル信号処理に必要な Python の機能 1

1. Python の開発環境 5
   - a. Anaconda のインストール 5
   - b. Spyder の起動と終了 5
   - c. Spyder の画面構成 6
     - i. エディタ 6
     - ii. IPython コンソール 6
     - iii. 変数エクスプローラー，ファイルエクスプローラー 7
2. 高性能関数電卓としての Python の利用 7
   - a. 数値と四則演算 8
   - b. 数値の変数への代入 9
   - c. 複素数と四則演算 10
   - d. ベクトルと行列 12
   - e. より本格的な関数の利用 16

f. 真偽値（論理値，ブール値）と論理演算 20

g. 文字列 21

h. 比較演算 21

i. 演算の優先順位 22

j. SpyderとIPythonコンソールの便利な機能 24

i. 変数エクスプローラー 24

ii. ファイルエクスプローラー 24

iii. ヒストリログ 24

iv. キーボードの矢印記号（方向キー ↑ ↓ ← →） 24

v. マジックコマンド 25

3. Pythonによるプログラミング 26

a. エディタによるプログラムの作成と実行 27

b. 制御構文 27

i. for 文 28

ii. while 文 29

iii. if 文 30

c. 関数の定義 32

i. 関数定義の例 32

ii. 関数の再帰的定義の例 33

d. グラフの描画 34

i. Pyplot インターフェースを用いたグラフの描画 35

ii. オブジェクト指向インターフェースを用いたグラフの描画 36

e. 関数のモジュールとパッケージ 41

i. 数値計算モジュール numpy 41

ii. 信号処理パッケージ signal 42

iii. 数学モジュール math 43

iv. Sys モジュール 44

v. 自作関数 44

vi. Python の組み込み関数 45

4. 参考資料 46

a. Python の入門 46

b. Python の詳細 46

c. その他 47

1. Python の開発環境

a. Anaconda のインストール

Python 対応 DSP に記載されている Python のプログラムを実行するために，Python のインストー ルとその統合開発環境を準備する．Python とその統合開発環境は，

Anaconda ディストリビューション：

https://www.anaconda.com/products/individual

より無償でダウンロード可能 である（2021 年 8 月現在）．

Anaconda ディストリビューションには，Python 本体だけではなく，ライブラリやエディタ，統合開発環境など，Python の実行に必要なソフト ウェア群が多く含まれている．Pyhotn 対応 DSP では，Anaconda ディストリビューションに 標準で含まれる統合開発環境 Spyder を用いている．

Anaconda をインストールしたら，Anaconda 版のコマンドプロンプト Anaconda Prompt において

```
> conda update --all
```

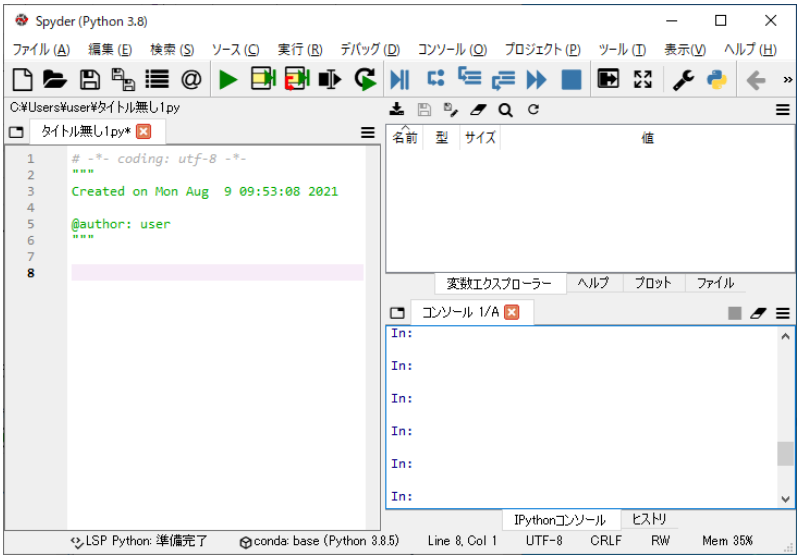
を実行して，ライブラリ群を最新版に更新しておくことが望ましい．Python 対応 DSP に記載されているプ ログラムのほとんどは Anaconda に標準で含まれるライブラリ群を用いて実行可能である． ただし，第12章「2 次元信号とフーリエ変換」および第13章「2次元ディジタルフィルタ」 においては画像処理関係のライブラリである OpenCV を追加でインストールする必要がある． Anaconda Prompt において

```
> conda install -c conda-forge opencv
```

を実行すると OpenCV がインストールされる．

b. Spyder の起動と終了

Spyder のアイコンをクリックすると次のような統合開発環境の画面が立ち上がる．



Spyder を終了するには画面の右上の「閉じる」ボタン×印をクリックする．あるいは，メニューバーの「ファイル(A)」をクリックし，その中のコマンド「終了(Q)」をクリックする．

c. Spyder の画面構成

Spyder の画面構成はデフォルトでは以下のようになっている．(表示の設定によって以下とは異なる画面が立ち上がるかもしれない)

i. エディタ

Spyder 画面の左側に「エディタ」ウィンドウがあり，ここで Python のプログラムの編集とデバッグ，実行等を行う．作成されたプログラムの実行とデバッグなどは，Spyder 画面上部のメニューバーのメニュー「実行(R)」と「デバッグ（D）」をクリックして行う．文法エラーや実行結果は，右下の「IPython コンソール」ウィンドウに表示される．またメニュー「ファイル（A）」により，ファイルの新規作成，保存と読み込み，印刷などを行うことができる．

ii. IPython コンソール

Spyder 画面の右下の画面は「IPython コンソール」ウィンドウである．IPython コンソールは，Python を対話型インタープリタとして利用する際にユーザーと

Python の間を取り持つ便利なインターフェースである．IPython コンソールには入力プロンプトが以下のように表示されている．

```
In[連番]:
```

ここに表示されている入力プロンプト In: にキーボードからコード（定数，変数，式，関数，コマンドなど）を入力すると，Python は入力されたコードを解釈して，その結果を出力プロンプト O t: に表示する（ただし結果を表示しないコードもある）．

```
Out[連番]:
（実行結果の表示）
```

この入力と出力の基本動作を繰り返すことで様々な計算を行うことができる．

入力として，定数，変数，式などの入力でもよいし，ファイルに保存（ファイル名.py で保存）されている複数行のプログラムのファイル名でもよい．

In: と Out: における整数は連番のプロンプト番号である．この番号をキーとして「ヒストリログ」ウィンドウで入力プロンプトへの過去の入力を確認できる．本資料では，プロンプト番号は省略している．本資料では IPython コンソールの入出力は灰色の背景中のゴシックで表されている．

iii. 変数エクスプローラー，ファイルエクスプローラー

Spyder の右上の画面には，「変数エクスプローラー」ウィンドウと「ファイルエクスプローラー」ウィンドウがある．「変数エクスプローラー」は定義されている変数の情報（型，サイズ，値など）を表示する．「ファイルエクスプローラー」は保存されているファイルの情報を表示し，削除，検索等に用いられる．

2. 高性能関数電卓としての Python の利用

Python の基本的な動作は対話型インタープリタである．IPython コンソールの入力プロンプト In: にコードを入力すると，Python の計算結果が出力プロンプトに表示される（計算結果が表示されないコードもあることに注意すること）．このような動作は電卓と同様の動作である．本章では Python を高性能関数電卓とみなし，数行の簡単なコードを入力し，その結果を見ることで Python の基本的な動作と文法を学ぶこととする．

なお多数のコードを一つ一つ入力して Python を使うことは効率が悪いので，このような場合には第３章に述べるようにエディタによりひとまとまりのプログラムを作成し，これを実行させる方がよい．

a. 数値と四則演算

Python で用いられる数値の表現と四則演算は数学や他のプログラミング言語の場合とほとんど同じであり，直感的に理解できるものである．数値に対しては，その型（整数，浮動小数点，複素数など）を意識せずに変数に代入して保存できる．

次に実際の例により Python における数値の表現と四則演算を見ていく．なお # は，これ以降から行末までがコメントであることを表す．コメントは行の先頭から入れてもよいし，行の途中から入れてもよい．以下のコメントは入出力の説明のために入れてあるものであり，実際に入力する必要はない．なお本資料では IPython コンソール画面の入出力は灰色で色付けされている．

以下に数値の表現と四則演算の例を示す．

```
In: 3                          # 整数
Out: 3
```

```
In: -3                         # 負の整数
Out: -3
```

```
In 3.14                        # 実数
Out: 3.14
```

```
In: 3.14e2                     # e2は10の2乗を表す． E2, e+2も同等である．
Out: 314.0
```

```
In: 3.14e-2                    # e-2は10の-2乗を表す． E-2でも同様である．
Out: 0.0314
```

```
In: (8 + 3 * 2) / (4 - 6)      # 四則演算
Out: -7.0
```

```
In: 5 / 2                      # 商を浮動小数点として出力
Out: 2.5
```

```
In: 5 // 2                    # 商(2.5)を整数2に切り捨て
Out: 2
```

```
In: 5 % 2                     # 剰余
Out: 1
```

```
In: 2** 4                     # べき乗（2の4乗）
Out: 16
```

```
In: abs(-3.14)                # 絶対値
Out: 3.14
```

b. 数値の変数への代入

数値と演算結果の変数への代入は変数の型を宣言する必要なく直接行える.

```
In: x = 2
In: y = 5
```

数値を変数に代入する際には数値は表示されない．数値を表示したいときには入力プロンプトに変数名を入れる.

```
In: x
Out: 2
```

```
In: y
Out: 3
```

一行の中に複数の代入の式を入力することができる.

```
In: x, y = 2, -4.5            # x = 2; y = -4.5 と書いてもよい.

In: x, y
Out: (2, -4.5)
```

変数の代入において一時変数を使わずに値を交換(swap)することができる.

```
In: x, y = y, x                    # 変数の値の交換

In: x, y
Out: (-4.5, 2)
```

変数の代入の右辺において複数の演算を行い，その結果を複数の変数に代入できる．

```
In: x, y  = x + y, abs(x -y)       # 演算後の代入

In: x, y
Out:（-2.5, 6.5)
```

c. 複素数と四則演算

複素数も宣言せずに利用できる．虚数単位は j または Jである．複素数は　実部 + 虚部 j と書く．複素数に対する四則演算も整数と実数の場合と同様に自然に記述できる．

```
In: 7j                             # 純虚数
Out: 7j
```

```
In: 3 + 7j                         # 複素数
Out: (3+7j)
```

```
In: complex(3, 7.3)                # 複素数を生成するために関数 complexを用いる．
Out: (3+7.3j)
```

```
In: 1 + 2j + 3 - 4j                # 複素数の和
Out: (4 -2j)
```

```
In: (1 + 2j) * (3 - 4j)            # 複素数の乗算
Out: (11+2j)
```

```
In: (1 + 2j) / (3 + 4j)            #複素数の除算
Out: (0.2+0.4j)
```

整数，実数（浮動小数点），複素数などの数値の型を意識することなく，変数に値を代入することができる．

```
In: x, y = 3 + 7j, -5 + 6j        # 複素数の代入と表示

In: x, y
Out: ((3+7j), (-5+6j))
```

```
In: v = x + y                     # 複素数の加算

In: v
Out: (-2+13j)
```

```
In: w = x * y                     # 乗算後の代入

In: w
Out: (-57-17j)
```

変数 w の実部と虚部，複素共役，絶対値は以下のように求められる．

```
In: w.real                        # 複素数wの実部
Out: -57.0
```

```
In: w.imag                        # 複素数wの虚部
Out: -17.0
```

```
In: w.conjugate()                 # 複素数wの共役
Out: (-57+17j):
```

```
In: abs(w)                        # 複素数wの絶対値
Out: 59.481089431852205
```

● 整数，実数，複素数に対する演算・操作等のまとめ

（x と y は整数，実数，複素数の変数）

算術演算	説明	備考・例外など
x + y	x に y を加算する	
x - y	x から　yを引く	
x * y	x にy を乗ずる	
x / y	x を y で割る	
x // y	x を y で割ったのち切り捨てる	複素数には適用されない

x ** y	x の y 乗	
x % y	x を y で割った余り	複素数には適用されない
x.real	x の実部	
x.imag	x の虚部	
x.conjugate()	x の複素共役	
abs(x)	x の絶対値	

d. ベクトルと行列

ベクトルと行列の定義のために数値計算モジュール numpy を読み込み（インポート），その中の関数 array を用いると便利である． numpy を読み込むときに，慣例的に以下のように別名 np を付けておく．

```
In: import numpy as np          # numpyのインポートと別名 npの指定
```

上記のように読み込むことで numpy の中の関数は np.関数名 として読み出して利用できるようになる．

以下のように数値計算モジュール numpy の関数 array によってベクトルが定義できる．

```
In: x = np.array([2, 4, 6])        # ベクトルの定義と代入

In: x
Out: array([2, 4, 6])
```

ベクトルの要素は, x[0], x[1] などで指定される．ここで注意しなければならないことは Python ではベクトルのインデックスは 0 から始まることである．

```
In: x[0]                           # ベクトルx のインデックスは0から始まることに注意
Out: 2

In: x[1]
Out: 4
```

ベクトルに対して以下のようにスカラー倍，和，差，要素ごとの積，要素ごとの商を計算できる．

```
In: x * 3                    #スカラーによる乗算
Out: array([ 6, 12, 18])
```

```
In: x / 2                    # スカラーによる除算
Out: array([1., 2., 3.])
```

```
In: y = np.array([2, 2, 3])  # もう一つベクトルを定義する
```

```
In: x + y                    # ベクトルの和
Out: array([ 4, 6, 9])
```

```
In: x – y                    # ベクトルの差
Out: array([0, 2, 3])
```

```
In: x * y                    # 要素ごとの積．数学的な積ではないことに注意．
Out: array([ 4,  8, 18])
```

```
In: x / y                    # 要素ごとの商．
Out: array([1., 2., 2.])
```

信号処理においてよく用いられるベクトルとして要素がすべて 1 あるいは 0 のベクトルがある．これらは以下のように 生成できる．

```
In: np.ones([3])             # すべての要素が 1 の1× 3ベクトル
Out:  array([1., 1., 1.])
```

```
In: np.zeros([2])            # すべての要素が0の1× 2ベクトル
Out: array([0., 0.])
```

行列も関数 array を用いて生成できる．また要素がすべて 1 および要素がすべて 0の行列も ones と zeoros を用いて生成でき，単位行列は eye を用いて生成できる．ベクトルの場合と同様に要素のインデックスは 0 から始まることに注意が必要である．

```
In. X = np.array([[1, 2, 3], [4, 5, 6]])  # 2x3 行列の定義

In: X
Out:
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
In: X[0,0]                    # 行列のインデックスは0から始まることに注意
Out: 1
```

```
In: U = np.ones([2, 3])       # すべての要素が1の2x3行列

In: U
Out:
array([[1., 1., 1.],
       [1., 1., 1.]])
```

```
In: Z = np.zeros([2, 3])      # すべての要素が0の2x3行列

In: Z
Out:
array([[0., 0., 0.],
       [0., 0., 0.]])
```

```
In: E = np.eye(3)             # 3x3 の単位行列

In: E
Out:
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

行列に対してスカラー倍，和，差，要素ごとの積，要素ごとの商を計算できる．

```
In: A = np.array([[2, 3], [4, 5]])      # 2×2行列の代入

In: B = np.array([[2, 3], [2, 1]])      # 2×2行列の代入
```

```
In: A                         # 確認
Out:
array([[2, 3],
       [4, 5]])
```

```
In: B                          # 確認
Out:
array([[2, 3],
       [2, 1]])
```

```
In: A + B                      # 行列の和
Out:
array([[4, 6],
       [6, 6]])
```

```
In: A – B                      # 行列の差
Out:
array([[0, 0],
       [2, 4]])
```

```
In: A * B                  # 行列の要素ごとの積（行列の積ではないことに注意）
Out:
array([[4,
       [8, 5]])
```

```
In: A / B                      # 行列の要素ごとの商
Out:
array([[1., 1.],
       [2., 5.]])
```

```
In: np.dot(A, B)               # 行列の積
Out:
array([[10,  9],
       [18, 17]])
```

- ベクトルと行列に対する演算・操作等のまとめ

（A と B は行列であり，以下の演算に対して適切な大きさを持っているものとする）

算術演算	説明	備考・例外など
A + B	加算	
A - B	減算	
A * B	要素ごとの乗算	

A / B	要素ごとの除算	
A // B	要素ごとに割ったのち切り捨て	
A ** B	要素ごとのべき乗	
A % B	要素ごとの剰余	
np.dot(A, B)	行列の積	

e. より本格的な関数の利用

整数，実数，複素数，ベクトル，行列に対してのより本格的な関数の利用について説明する．ここでは，代表的な数値計算モジュール numpy を利用する．numpy を読み込むときに慣例的に別名 np を付ける．関数モジュール numpy の詳細についてはプログラミングの章で再度説明する．

最も重要な定数である円周率（π = 3.14159…）とネピアの定数 (e = 2.71828…) を以下のように関数により作り出すことができるので，これを直接にあるいは変数に代入して利用できる．

```
In: np.pi                    # 円周率の生成
Out: 3.141592653589793
```

```
In: np.e                     # ネピアの定数（自然対数の底）の生成
Out: 2.718281828459045
```

numpy には数学でしばしば用いられる絶対値，偏角，三角関数，対数関数などの重要な関数が準備されている．

```
In: x, y, z = -2, 3, 1 + 2j
```

```
In: np.abs(x)                # 絶対値
Out: 2
```

```
IIn: np.sqrt(y)              # 平方根
Out: 1.7320508075688772
```

```
IIn: np.abs(z)               # 複素数の絶対値
Out: 2.23606797749979
```

```
In: np.angle(z)              # 複素数の偏角（位相）（単位は rad）
```

```
Out: 1.1071487177940904
```

```
In: np.log10(y)                    # 常用対数
Out: 0.3010299956639812
```

```
In: np.log(y)                      # 自然対数
Out: 1.0986122886681098
```

```
In: np.exp(1.0986122886681098)      # 指数関数
Out: 3.0000000000000004
```

```
In: np.exp(z)                      # 指数関数の引数に複素数を入れられる
Out: (-1.1312043837568135+2.4717266720048188j)
```

```
In: np.sin(np.pi / 4)              # 正弦関数（入力引数の単位は rad）
Out: 0.7071067811865476            # その他に，三角関数として, cos, tan,
                                   # arcsin, arccos, arctan, arctan2 などがある．
```

numpy の関数はベクトルと行列にもそのまま適用できる．ベクトルあるいは行列を関数の入力引数とすると，その要素ごとに関数が適用されて出力引数が計算される．出力引数の大きさは入力引数の大きさと同じ大きさである．この機能により繰り返しのループが不要になり，プログラミングが容易となる．

```
In: w = np.array([0, 1, 2, 3])

In: np.sqrt(w)                     # ベクトルに対する平方根
array([0.        , 1.        , 1.41421356, 1.73205081])
```

```
In: np.sin(w * np.pi / 4)          # ベクトルに対する正弦関数の適用
Out: array([0.        , 0.70710678, 1.        , 0.70710678])
```

```
In: 2.0 ** w                       # べき乗
Out: array([1., 2., 4., 8.])
```

```
In: A = np.array([[1, -2], [-3, 4]])      # 行列の定義

In: A
Out:
```

```
array([[1, -2],
       [-3, 4]])
```

```
In: np.abs(A)                    # 行列の要素ごとの絶対値
Out:
array([[1, 2],
       [3, 4]])
```

行列に対する代数的な演算を以下に示す.

```
In: A.T                          # 転置
Out:
array([[ 1, -3],
       [-2,  4]])
```

```
In: np.linalg.det(A)             # 行列式
Out: -2.0000000000000004
```

```
In: np.linalg.inv(A)             # 逆行列
Out:
array([[-2. , -1. ],
       [-1.5, -0.5]])
```

```
In: r, V = np.linalg.eig(A)      # r に固有値，行列 Vに固有ベクトルを代入

In: r
Out: array([-0.37228132,  5.37228132])

In: V
Out:
array([[-0.82456484,  0.41597356],
       [-0.56576746, -0.90937671]])
```

```
In: B = np.array([[2,3],[4,5]])        # もう一つの行列を定義

In: B
Out:
array([[2, 3],
```

```
        [4, 5]])
```

```
In: np.dot(A, B)                # 行列の積
Out:
array([[-6, -7],
        [10, 11]])
```

```
In: b = np.array([[1], [-5]])  # 2x1行列の定義
In: np.linalg.solve(A, b)      # 連立方程式 Ax = b の解法
Out:                           # 連立方程式の解
array([[3. ],
        [1. ]])
```

- 高性能関数電卓における関数のまとめ

（関数モジュール numpy を別名 np として読み込んで関数を利用する）

定数	説明	備考・例外など
pi	円周率	
e	ネピアの定数（自然対数の底）	
算術演算	説明	備考・例外など
sqrt	二乗根	
abs	絶対値	
angle	複素数の偏角	
三角関数	説明	備考・例外など
sin，cos， tan	正弦， 余弦， 正接の関数	
arcsin, arccos, artcan，arctan2	正弦， 余弦， 正接の逆関数	
対数， 指数関数	説明	備考・例外など
log，log10, log2	自然対数， 常用対数， 底2の対数	
exp	指数関数	
行列演算	説明	備考・例外など
dot	行列の積	
linalg.inv	逆行列	
linalg.solve	連立方程式の解	
linalg.eig	固有ベクトルと固有値	

f. 真偽値（論理値，ブール値）と論理演算

Python には真偽値（論理値，ブール値）として, 真 True と 偽 Falseがある．真偽値に対して論理和 or, 論理積 and，否定 not などの演算を行うことができる．真偽値は if 文や while 文などにおける条件に用いられる．

```
In: True                    # 真
Out: True
```

```
In: False                   # 偽
Out: False
```

```
In: A = True                # 真偽値の変数への代入
In: B = False
```

```
In: A                       # 代入の確認
Out: True
```

```
In: B                       # 代入の確認
Out: False
```

```
In: not A                   # 論理否定
Out: False
```

```
In: A and B                 # 論理積
Out: False
```

```
In: A or B                  # 論理和
Out: True
```

```
In: (A and (not B)) or ((not A) and B)        # 排他的論理和
Out: True
```

- 論理演算のまとめ

(変数 A と B は真偽値)

論理演算	True を返す条件	備考

A and B	A と Bが双方とも True のとき	
A or B	A と Bうち少なくとも一つが Trueのとき	
not A	A が Falseのとき	

g. 文字列

文字列は シングルクォーテーションまたはダブルクォーテーションで囲まれた文字の並びである．

```
In: 'Hello'                    # 文字列
Out: 'Hello'
```

```
In: H = 'Hello'                # 文字列の代入

In: sp = ' '                   # 空白一つの代入

In: W = 'World'                # 文字列の代入
```

```
In: HW = H + sp + W            # 文字列の連結

In: HW
Out: 'Hello World'
```

h. 比較演算

二つの数値，ベクトル，行列，文字列などの比較のために以下の比較演算子がある．比較演算の結果は True あるいは False になる．後にのべる if 文や while 文などの制御構文の条件において比較演算と真偽値は重要である．

```
In: x, y = 4, 2
```

```
In: x == 2 * y                 # 等しい
Out: True
```

```
In: x >= 3 * y                 # より大きいか等しい
Out: False
```

```
In: v = np.array([3, 4, 5])

In: w = np.array([4, 5, 5])
```

```
In: v >= w                          # ベクトルに対して要素ごとの比較
Out: array([False, False,  True])        # 要素ごとの比較結果.
```

```
In: T, F = True, False              # 論理値の真と偽の代入
```

```
In: T == F                          # 等しい
Out: False
```

```
In: T != F                          # 等しくない
Out: True
```

- 比較演算のまとめ
  （多くの場合，比較演算の対象 x と y は整数，実数，複素数，ベクトルあるいは行列をとりうるが，比較演算子によって適用可能な対象が異なることに注意せよ）

比較演算子	Trueを返す条件	備考
x == y	x は y に等しいとき	
x != y	x は y に等しくないとき	
x > y	x は y より大きいとき	複素数には適用されない
x >= y	x は y より大きいか等しいとき	複素数には適用されない
x < y	x は y より小さいとき	複素数には適用されない
x <= y	x は y より小さいか等しいとき	複素数には適用されない

i. 演算の優先順位

一つの式に多数の演算があるとき，Python は，あらかじめ決められている演算の優先順位に従って演算を実行する．式の中に( )があればその中の演算をまず優先して，下に示す演算の優先順位に従って式を評価する．同一の優先順位の演算が続く場合にはより左側の演算を優先する．

優先順位	演算	演算の説明
1	**	べき乗
2	+x, -x,	正数，負数

3	*, /, //, %	乗算，除算，整数除算，剰余
4	+, -	加算，減算
5	<, <=, >, >=, !=, ==	比較演算子
6	not	論理否定
7	and	論理積
8	or	論理和

補足：　べき乗演算子 ** は右側にある単項算術演算子あるいは単項ビット演算子より弱い結合優先順位になる．例えば 2**-1 は 0.5 である．

以下の算術式と論理式は形式上の誤りはないが，演算順序がわかりにくい例である．

```
In: 8 / 2 * (2 + 2)                # 少しわかりにくい算術式
Out: 16.0
```

```
In: 1 - 2 * 3 / - 4 ** - 5 + 6     # わかりにくい算術式
Out: 6151.0
```

```
In: T, F, x, y = True, False, 1, 2
```

```
In: T or x >= y and not F          # わかりにくい論理式
Out: True
```

上記の複雑な算術式と論理式は形式的には間違ってはいないが，わかりにくく，誤解されるため，以下のように ( ) を付けて優先順位を明示するか，いくつかの式にわけるべきであろう．

```
In: (8 / 2) * (2 + 2)
Out: 16.0
```

```
In: 1 - (2 * 3) / ( -(4 ** (-5)) ) + 6
Out: 6151.0
```

```
In: T or ( (x >= y) and (not F) )
Out: True
```

もちろんわかりにくい式や誤解されやすい式は書くべきではない．

j. SpyderとIPythonコンソールの便利な機能

次章のプログラミングに入る前に，Spyder とIPython コンソールの便利な機能を紹介しておく．これらは Python を対話型インタープリタ（高性能関数電卓）として利用するときや，第３章での複雑なプログラミングを行う際に大いに助けになる．

i. 変数エクスプローラー

入力プロンプト In: に何度も入力を続けて計算を継続していると，どのような変数が定義され，どんな値をもっているのか分からなくなることがある．そのときには変数エクスプローラーを用いると，定義されている変数の情報（変数名，型，サイズ，値）を知ることができる．

ii. ファイルエクスプローラー

ファイルエクスプローラーはファイラーであり，保存されているファイルの情報を表示し，削除，検索等に用いることができる．

iii. ヒストリログ

ヒストリログを表示するとPython コンソールへの入力の履歴を見ることができる．

iv. キーボードの矢印記号（方向キー ↑ ↓ ← →）

IPython コンソールの入力プロンプト In: において表示されている数値や式などを修正する際に，キーボードの左矢印キー ← と右矢印キー → でカーソルの移動ができるので，適当なところにカーソルを移動して文字の挿入と削除ができる．

方向キー ↑ を押すと一つ前の入力コードが表示されるので，一つ前の入力を再度使うことができる．方向キー ↑ を押した数だけ前の入力コードが表示され，方向キー ↓ で一つもとに戻る．

左矢印キー ← と右矢印キー → でカーソルを移動して，文字の挿入と削除をすることで，一つ前の入力コードを修正して使えば，キー入力の手間をはぶくことができる．

v. マジックコマンド

以下に示す様々なコマンドは IPython のマジックコマンドと呼ばれ，入力プロンプト In: に入力して用いる．Python を高性能関数電卓として用いるときにも，第 3 章で説明するプログラムの作成のためにも役にたつ．

1） who, whos：定義されている変数名等を表示

```
In: who                                  # 定義されている変数名等を表示できる
A       N       no      np      y       z
```

```
In: whos                                 # 変数の情報を詳しく表示
Variable   Type       Data/Info
-------------------------------
A          ndarray    3: 3 elems, type `int32`, 12 bytes
N          int        4
np         module     <module 'numpy' from 'C:¥<...>ges¥¥numpy¥¥__init__.py'>
y          float      3.5
z          complex    (3+5j)
```

2） reset, reset_selective: 定義されている変数等の消去

```
In: reset                                # 定義された変数等をすべて一括で消去
In: reset_selective 変数名               # 指定した変数等を消去
```

3） history: 入力履歴を表示

```
In: history
    （表示を省略）
```

4） exit, quit: 現在のコンソールを閉じて新しいコンソールを表示

入力プロンプト In: に何度も入力を続けて計算を続けると，定義されている変数が増え，入力の履歴が複雑になってしまうことがある．このようなときには現在のコンソールを閉じて新しいコンソールを開くとよい．
このために exit または quit を用いる．

```
In: exit
```

```
In: quit
```

5） run, rerun: Pythonのプログラムを実行

```
In: run ファイル名       # Pythonプログラム（ファイル名.pyで保存）を実行
```

```
In: rerun                    # 直前のプログラムを再度実行
```

6)　pwd, ls, pycat：その他の便利なコマンド
　　その他の便利なコマンドとして以下のものがある．

```
In: pwd                      # カレントディレクトリの表示
Out: 'C:¥¥マイドライブ'
```

```
In: ls                       # ディレクトリ名やファイル名等を表示
 ドライブ C のボリューム ラベルは xxxx です
 ボリューム シリアル番号は ---- - ---- です

 C:¥ yyyy のディレクトリ

2021/08/19  08:36    <DIR>          .
2021/08/19  08:36    <DIR>          ..
    （省略）
2021/08/07  17:17              474 example1.py
                  17 個のファイル          3,022,581 バイト
                   6 個のディレクトリ  263,646,572,544 バイトの空き領域
```

```
In: pycat example1.py            # Pythonのプログラムのテキスト表示
    （example1.pyの内容が表示されるが，省略する）
```

3. Pythonによるプログラミング

　第２章では Python を高性能関数電卓として利用して，Python の簡単な文法と関連する関数について説明してきた．高性能関数電卓としての利用はコンソールにコード（式や数値，関数など）を一つずつ入力するため，計算の結果がすぐに得られて便利であるものの，多数のコードの入力が必要になる複雑な計算を行うには不向きである．複雑な計算を行うためにはエディタによってひとまとまりのプログラムを作成してファイルに保存し，これを実行することが必要である．
　以下の説明ではエディタウィンドウの中のプログラムを水色の背景で表し，プログラムを実行したときの IPython コンソールの表示を灰色の背景で表している．

a. エディタによるプログラムの作成と実行

統合開発環境 Spyder のエディタ画面にキーボードから以下のようなプログラムを入れて，エディタによるプログラムの作成と実行の流れを追いかけてみる．行番号(1:, 2: …)は説明用のものなので入力しない．

```
1:    # プログラム例 example_1.py
2:    import numpy as np                    # numpy のインポート
3:
4:    radius = 3.0                          # 円の半径 r
5:    area = np.pi * radius**2              # 円の面積の計算 π r**2
6:    print('radius = ¥n', radius)          # 半径 radius の印字
7:    print('area = ¥n', area)              # 面積area の印字
```

次に Spyder のメニュー「ファイル(A)」をクリックし，このプログラムにファイル名（拡張子は.py）をつけて保存する．ここでは，example_1.py として保存する．Spyder のメニュー の「実行(R)」をクリックして実行する．IPythonコンソールにエラーメッセージが出たら，メッセージを読んで，エディタ画面でプログラムを適切に修正して，再度，保存して，実行する．エラーがなければ以下のような計算結果が表示される．

```
radius =
 3.0
area =
 28.274333882308138
```

IPython コンソールから実行する場合にはマジックコマンドの run を以下のように用いる．

```
In: run  example_1
```

b. 制御構文

ディジタル信号処理では繰り返し処理と条件分岐が用いられる．しばしば用いられる繰り返し処理は for 文と while 文であり，条件分岐は if 文である．これらの制御構文を用いることで複雑な計算を実行できる．

i. for 文

for 文の構文は以下のようである．

```
for [変数名] in [イテラブル]：
    [コードブロック]
```

ここで変数名は自由につければよい．イテラブルは数値の並びあるいは文字列の並びなどであり，その要素が一つずつ変数に代入されるたびにコードブロックが実行される．コードブロックは，空白 4 ヶでインデントされた一つ以上のコードの集まりである．ここで重要なことは，空白 4 ヶのインデントはコードブロックが繰り返しの対象となるための文法上必須のインデントである．この必要性により，Python では for 文において（次に示す while 文と if 文においても）強制的にインデントが行われることになり，プログラムの可読性が高まる．

for 文の例を以下に示す．

```
1:    # プログラム例 example_2.py
2:    # ライプニッツの公式π = 4(1 - 1/3 + 1/5 – 1/7 + 1/9 - … )
3:
4:    print('ライプニッツの公式による π の近似値')
5:    N = 10**3
6:    S = 0
7:    for n in range(0, N + 1):
8:        p = 1 / (2 * n + 1)
9:        S = S + (-1)**n * p
10:   pi = 4 * S
11:   print('pi = ¥n', pi)
```

```
ライプニッツの公式による π の近似値
pi =
 3.1425916543395442
```

7: においてrange 関数は指定された範囲の整数を生成する．range(1, N＋1）は，0, 1, 2, 3, …, N の整数を生成し，これが順番に n の値となり，n の値ごとにコードブロックが実行される．
8-9: がコードブロックである．

for 文が二重に必要であれば，以下のように for 文を入れ子にして用いる．

```
1:    # プログラム例　example_3.py
2:    # ライプニッツの公式 π = 4(1 - 1/3 + 1/5 – 1/7 + 1/9 - … )
3:
4:    print('ライプニッツの公式による π の近似値')
5:    M = 6
6:    for m in range(0, M + 1):
7:        N = 10**m
8:        S = 0
9:        for n in range(0, N + 1):
10:           p = 1 / (2 * n + 1)
11:           S = S + (-1)**n * p
12:       pi = 4 * S
13:       print('N = ', N, '    pi =', pi)
```

```
ライプニッツの公式による π の近似値
N =  1     pi =  2.666666666666667
N =  10     pi =  3.232315809405594
N =  100     pi =  3.1514934010709914
N =  1000     pi =  3.1425916543395442
N =  10000     pi =  3.1416926435905346
N =  100000     pi =  3.1416026534897203
N =  1000000     pi =  3.1415936535887745
```

ii. while 文

while 文は条件が True である間，コードブロックを繰り返す文であり，以下のような構文をもつ．

```
while [条件] :
    [コードブロック]
```

以下にwhile文の例を示す．

```
1:   # プログラム例 example_4.py
2:   x = 2
3:   while  x < 10:
```

```
4:      print('x = ', x)
5:      x = x**2  - 1
```

```
x =  2
x =  3
x =  8
```

3: において x < 10 の比較演算の結果がTrue の間,
4-5: のコードブロックを繰り返し実行する．比較演算の結果が False であれば while 文を終了する．

iii. if 文

if 文は一般的に以下のような構文をもつ． ここで elif は，else if の意味である．if 文において elif あるいは else がない場合もある．

```
if [条件式1] :
    [コードブロック 1 ]
elif [条件式2] :
    [コードブロック2]
else:
    [コードブロック3]
```

上記のif 文においては
if において [条件式 1] が True であれば，[コードブロック 1] を実行して if 文を終了する．False であれば elif に行く．
elif において [条件式2] が True であれば，[コードブロック2] を実行して if 文を終了する．False であれば else に行く．
else において [コードブロック3] を実行して if 文を終了する．

if 文の例（実行するコードブロックが一つの場合）を示す．

```
1:  # プログラム例　example_5.py
2:
3:  x = 4
4:  if x % 2 == 0 :
5:      print('x = ', x)
6:      print('x is even.')
```

```
x =  4
x is even.
```

4：において 条件式 x % 2 == 0 は True であるので,
5-6: のコードブロックが実行される.

if-else 文の例（実行するコードブロックが二つの場合）を示す.

```
1:    # プログラム例　example_6.py
2:
3:    import numpy as np
4:    x = -2
5:    print('x =', x)
6:    if  x >= 0:
7:        root_x = np.sqrt(x)
8:    else:
9:        root_x = np.sqrt(-x) * 1j
10:   print('root_x =', root_x)
```

```
x = -2
root_x = 1.4142135623730951j
```

6：において条件式 x >= 0 は Flase であるので,
8: の else に進み,
9: の root_x = np.sqrt(-x) * 1j を計算して，if-else 文は終了し,
10: において root_x を印字する.

if-elif-else 文の例（実行するコードブロックが三つの場合）の以下に示す.

```
1:    # プログラム例  example_7.py
2:    import numpy as np
3:
4:    a, b = 2, 3                                    # 方程式 a*x + b = 0 の係数
5:    if (a == 0) and (b == 0):                      # 不定の場合
6:        print('a == 0, b == 0, Indeterminate equation.')
7:    elif (a == 0) and (b != 0):                    # 不能の場合
8:        print('a == 0, b != 0, Impossible equation.')
9:    else:                                          # 解が存在する場合
10:       print('a != 0')
```

```
11:         x = -b / a
12:         print('x = ¥n', x)
```

```
a != 0
x =
 -1.5
```

5: の条件は False であるから，
7: の elif に進み，この条件も False であるから，
9: の else に進み，
10-12: のコードブロックを実行して，こ の if-elif-else 文は終了する．

c. 関数の定義

同一の処理を複数回実行する場合や複雑で長い処理を実行する場合などは，これらの処理をひとまとまりの関数として定義して，これを呼び出して利用することにより，プログラミングが容易になり，可読性や保守性も高まる．関数の定義のために def 文を用いて，以下のように関数を定義する．ここでも関数の定義の範囲は先頭 4 ケの空白をもつコードブロックである．

```
def　関数名(入力引数1 , 入力引数２ , …, 入力引数N)：
    ［コードブロック］
```

もし定義したい関数に入力引数が必要なければ，def 関数名()　として定義する．なお関数内の変数は特に指定がなければ関数内だけで利用できるローカル変数である．

関数を定義するプログラムが完成したら，関数名.py のファイルとして保存する（ただし Python においては関数名とファイル名は一致させる必要はない）．ファイルとして保存されている関数をプログラムから呼び出して利用するためには，以下のようにプログラム中で関数を読み込でおく必要がある．

```
from <モジュール名>  import <オブジェクト名 または 関数名>
```

i. 関数定義の例

次に辺の長さが a, b, c である三角形の面積を求めるヘロンの公式を関数 heron として定義する例を示す．以下のプログラムをエディタにより作成し，heron.py として保存する．
ただし Python においては関数名とファイル名は一致させる必要はない．

```
1:  # ヘロンの公式 heron.py
2:  import numpy as np
3:
4:  def heron(a, b, c):
5:      s = (a + b + c) / 2
6:      area = np.sqrt(s * (s - a) * (s - b) * (s - c))
7:      return area
```

7: においてreturn area は変数 areaを関数 heron の戻り値（出力引数）とすることを表す．関数の定義において戻り値が不要であれば，return を入れる必要はない．

定義された関数 heron の利用の例を次に示す．

```
1:  # 関数 heron.pyの利用の例
2:  from heron import heron
3:
4:  x, y, z = 3, 4, 5
5:  S = heron(x, y, z)
6:  print('S =', S)
```

```
S =  6.0
```

2: において heron を読み込んでから，
5: において heron を利用する．

ii. 関数の再帰的定義の例

次の例は関数の再帰的定義（自分自身を呼び出すことによる定義）の例である．ここでは n の階乗に関しての再帰的性質 n! = n*(n-1)! を利用して階乗の関数 myfact を再帰的に定義している．

```
1:  # 階乗の再帰的定義 myfact.py
2:
3:  def myfact(n):
```

```
4:      if n == 0:
5:          return 1
6:      else:
7:          return n * myfact(n - 1)
```

4-5:において入力引数 n が 0 ならば 1を返し，入力引数 n が 0 ではない場合には 6-7:において n-1 の階乗 myfact(n-1) を再帰的に呼び出し，これに n を乗じて，n * myfact(n - 1) を戻り値としている．

```
1:  # 関数 myfactの利用の例
2:  from myfact import myfact
3:
4:  n = 5
5:  myfact(n)
```

```
120
```

2: において myfact を読み込んでから，
5: において myfact を利用する．

d. グラフの描画

グラフの描画のために以下のように描画モジュール matplotlib.pyplot を用いる．プログラムでは利用に先立って以下のように matplotlib.pyplot を読み込み，別名 plt を付けておく．

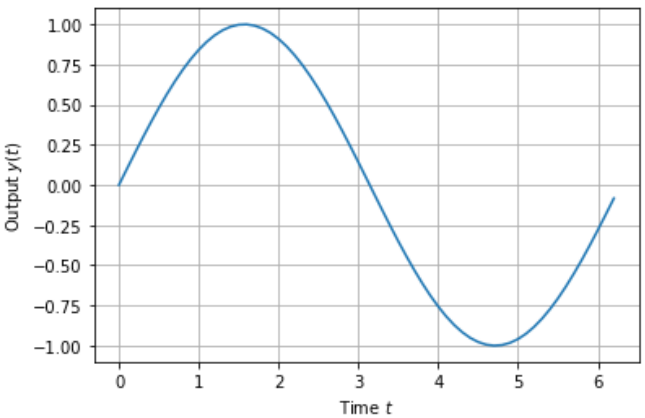
```
import matplotlib.pyplot as plt
```

matplotlib.pyplot を用いてグラフを描画する方法として二つの方式（インターフェース）がある．一つは Pyplot インターフェースを用いる方式である．この方式は Figure オブジェクト等を明示せずに簡易にグラフを描画できる．いわば MATLAB 流のグラフの描画である．もう一つはオブジェクト指向インターフェースを用いるものである．この方式では Figure オブジェクト等を明示して描画する．いくぶん手数はかかるが，詳細な条件指定により精緻な描画ができるので，Python 対応 DSP ではこの方式を採用している．

i. Pyplot インターフェースを用いたグラフの描画

この方法は後述の方法に比べて容易にグラフを描くことができるが，グラフの精緻な描画には適さない．以下のプログラムは関数 plot により正弦波のグラフを描く例である．

```
1:      # Pyplotインターフェースを用いたグラフの描画 example_8.py
2:      import numpy as np
3:      import matplotlib.pyplot as plt
4:
5:      t = np.arange(0, 2*np.pi, 0.1)
6:      y = np.sin(t)
7:      plt.plot(t, y)
8:      plt.grid(True)
9:      plt.xlabel('Time $t$'); plt.ylabel('Output $y(t)$')
10:     plt.show()
```



3: において matplotlib.pyplot を読み込み，別名 plt を付ける．

5-6: において刻み 0.1で，0〜2π までの時刻のベクトル t を生成し，y = sin(t) を計算する．

7: において plot(t, y) により，横軸 はベクトル t，縦軸は ベクトルy としてグラフを描く．ここで t と y のベクトルの長さ（要素数）は一致していなければならない．

8-9: において grid によりグラフに格子を入れ， xlabel により x 軸のラベルを文字列 'Time t' として関数 ylabel により y 軸のラベルを 'Output $y(t)$' とする．ここで $ $ はその中の文字列を数式モードで表すことを意味する．数式モードではフォント，上付き，下付き，特殊記号などが指定できる．

10: において7-9: で描いたグラフを show() によって実際に表示する．

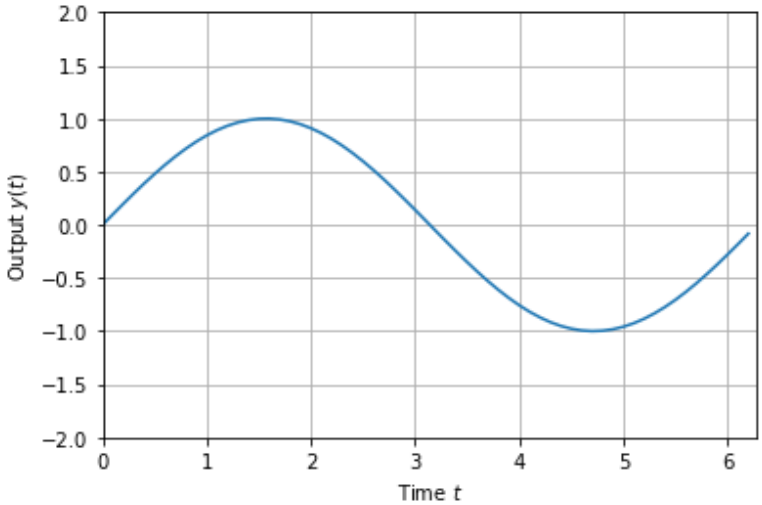
注意：7-9: はグラフの表示の準備段階であり，10:　の show() によって初めてグラフが実際に表示される．統合開発環境 Spyder では show() を実行しなくても図が表示されるようになっているため，Python 対応 DSP のプログラムでは show() を省略している．

ii. オブジェクト指向インターフェースを用いたグラフの描画

1) グラフの描画の基本

以下のプログラムは Figure オブジェクト等を明示してグラフを描画するプログラムの例である．

```
1:      # オブジェクト指向インターフェースを用いたグラフの描画
2:      # example_9.py
3:      import numpy as np
4:      import matplotlib.pyplot as plt
5:
6:      t = np.arange(0, 2*np.pi, 0.1)
7:      y = np.sin(t)
8:      fig = plt.figure()
9:      ax = fig.add_subplot(1, 1, 1)
10:     ax.plot(t, y)
11:     ax.set_xlim(0, 2*np.pi); ax.set_ylim(-2, 2)
12      ax.grid()
13:     ax.set_xlabel('Time $t$')
14:     ax.set_ylabel('Output $y(t)$')
15:     plt.show()
```

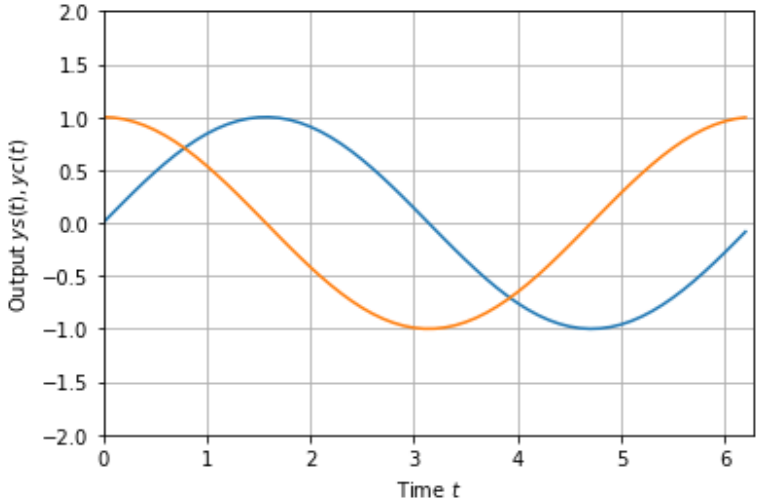


3: において 描画モジュール matplotlib.pyplot を読み込み，別名 plt を付ける．
8: において plt.figure メソッド により Figre オブジェクトを生成し, 変数 fig に代入する（fig と名付ける）. Figure オブジェクトはグラフや文字など全てが描かれる描画領域である．括弧()の中で様々なパラメータを指定することができるが，ここに何もパラメータを入れていないので，デフォルトの描画領域が確保される．
この段階で白紙１枚が用意され，その名称を fig としたと考えればよい．
9: 次に描画領域 fig の中に実際のグラフを描画するために使うAxes オブジェクトを生成する．（Axes とはAxis（座標軸）の複数形である）このために fig.add_subplot メソッドにより Axes オブジェクトを生成し，変数 ax に代入する（ax と名付ける）．パラメータ(1, 1, 1) は描画領域 fig を１行１列に分割し（すなわちこの場合には描画領域全体を使い），その１番目（この場合には描画領域全体）を使うこと意味する．これにより白紙の描画領域 fig に座標軸をもつグラフ用紙ax が作られたと考えればよい．
10: ax.plot(t, y) メソッドによりグラフ用紙 ax に，t を横軸， y を縦軸とする連続グラフをプロットする．ここで t と y のベクトルの長さ（要素数）は同一でなければならない
11-14: ではグラフ用紙 ax に様々な設定を行う．まず，
11: では ax.set_xlim メソッド，ax.set_ylim メソッドにより縦軸と横軸の範囲を指定する．
12:では ax.grid() メソッドによりグラフに格子を入れる．ここでカッコの中で格子の性質を指定できるが，ここでは () によりデフォルトの指定をしている．
13-14: では ax.set_xlabel メソッド, ax.set_ylabel メソッドにより横軸と縦軸のラベルを付けている．なお n と $y[n]$ のように $ $ で囲まれた文字は Tex, Latex で用いられている数式モードの指定である．数式モード $ $ の中ではフォント，上付き，下付き，特殊記号などが指定できる．
15: plt.show() によってグラフが実際に表示される．

2) 複数のグラフを重ねて描画

一つの描画領域 fig に複数のグラフを重ねて描画する場合には以下のプログラムのようにグラフ用紙 ax にグラフ ys を描き，その後 グラフ yc を描く．

```
1:    # 複数のグラフを重ねて描く example_10.py
2:    import numpy as np
3:    import matplotlib.pyplot as plt
4:
5:    t = np.arange(0, 2*np.pi, 0.1)
6:    ys = np.sin(t)
7:    yc = np.cos(t)
8:    fig = plt.figure()
9:    ax = fig.add_subplot(1, 1, 1)
10:   ax.plot(t, ys)
11:   ax.plot(t, yc)
12:   ax.set_xlim(0, 2*np.pi); ax.set_ylim(-2, 2)
13:   ax.grid()
14:   ax.set_xlabel('Time $t$'); ax.set_ylabel('Output $ys(t), yc(t)$')
15:   plt.show()
```



8: において描画領域 fig を確保する．

9: においてグラフ用紙 ax を確保する．

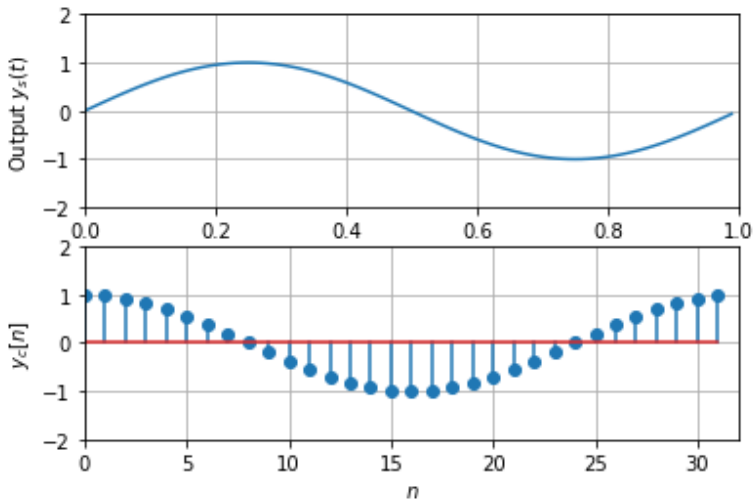
10: においてグラフ用紙 ax に plot(t, ys) により正弦波 ys を描画している．

11: において同じグラフ用紙 ax に plot(t, yc) により余弦波 yc のグラフを重ねて描画している．

3) 複数のグラフを別々に描画

一つの描画領域に複数のグラフを別々に描画するためには，以下のプログラムのように描画領域 fig に二つのグラフ用紙 ax1 とax2 を確保して，それぞれのグラフ用紙に描画する．

```
1:    # 複数のグラフを別々に描く example_11.py
2:    import numpy as np
3:    import matplotlib.pyplot as plt
4:
5:    t = np.arange(0, 1, 0.01)
6:    ys = np.sin(2 * np.pi * t)
7:    n = np.arange(0, 32)
8:    yc = np.cos((2 * np.pi / 32) * n)
9:    fig = plt.figure()
10:   ax1 = fig.add_subplot(2, 1, 1)
11:   ax1.plot(t, ys)
12:   ax1.grid()
13:   ax1.set_xlim(0, 1); ax1.set_ylim(-2, 2)
14:   ax1.set_xlabel('Time $t$'); ax1.set_ylabel('Output $y_s(t)$')
15:   ax2 = fig.add_subplot(2, 1, 2)
16:   ax2.stem(n, yc)
17:   ax2.grid()
18:   ax2.set_xlabel('$n$'); ax2.set_ylabel('$y_c[n]$')
19:   ax2.set_xlim(0, 32); ax2.set_ylim(-2, 2)
20:   plt.show()
```



9:において描画領域 fig を確保する．

10-14:において add_subplot メソッドにより描画領域 fig にグラフ用紙 ax1 を確保し，連続時間の正弦関数を描画する．パラメータ(2, 1, 1)は描画領域 fig を2行１列に分割し，その1番目（上側）を意味する．これにより描画領域 fig の上部にグラフ用紙 ax1 が作られたと考えればよい．
15-19:において add_subplot メソッドにより描画領域 fig にグラフ用紙 ax2 を確保し， 離散時間の余弦関数を描画する．パラメータ(2, 1, 2)　は描画領域 fig を 2 行 1 列に分割し，その2番目（下側）を意味する．これにより描画領域 fig の下部にグラフ用紙 ax2 が作られたと考えればよい．

- matplot.lib.pyplot のメソッドのまとめ

オブジェクト	メソッド	説明
Figure オブジェクト	plt.figure	Figure オブジェクトを生成
	plt.show	Figureオブジェクトを表示
	add_subplot	Figure オブジェクトに Axes オブジェクトを生成
	tight_layout	図の中にサブプロットがうまくおさまるようにサブプロットのパラメータを調整
Axes オブジェクト	plot_surface	３次元のサーフェスプロット
	plot	x-yプロット
	stem	ステムプロット（茎状のプロット）
	plot_wireframe	３次元のワイアフレームプロット
	pcolor	擬似カラープロット
	set_xlabel	x 軸のラベルを設定
	set_ylabel	y 軸のラベルを設定
	set_zlabel	z 軸のラベルを設定
	set_xlim	x 軸の表示限度を設定
	set_ylim	y 軸の表示限度を設定
	set_zlim	z 軸の表示限度を設定
	grid	格子を描画
	view_init	仰角と方位角を設定
	legend	グラフの凡例の位置を設定

e. 関数のモジュールとパッケージ

Python 自体には基本的な関数が用意されているが，数学的に高度な関数や特定応用分野の関数を利用する際には Python の拡張モジュールを用いる．Python 対応 DSP では拡張モジュールとして数値計算モジュール numpy，数学モジュール math，科学技術計算ライブラリ SciPy の関数を用いている．とくに SciPy の信号処理パッケージ signal 中の多数の信号処理用の関数をしばしば用いている．拡張モジュールとして提供されていない信号処理用関数は自作している．

i. 数値計算モジュール numpy

Python 対応 DSP では数値計算モジュール numpy の多数の関数を利用している． numpy の関数を利用するために以下のように読み込み，別名 np を付ける．

```
import numpy as np
```

実際に関数を利用する際には np.関数名 として利用する．Python 対応 DSP において用いている numpy の関数を以下にまとめている．

種類	関数名	説明
重要な定数	e	ネピアの定数(2.71828…), 自然対数の底
	pi	円周率　(3.14159 …)
数値計算, 三角関数, 対数など	abs	絶対値
	angle	複素数の偏角，出力引数の単位は rad
	cos	余弦関数，入力引数の単位は rad
	exp	指数関数，複素数の引数も可
	log10	常用対数
	log	自然対数
	max	配列の要素の最大値
	min	配列の要素の最小値
	prod	配列の要素の積
	roots	多項式の根
	sin	正弦関数，入力引数の単位は rad
	sinc	カーディナルサイン関数
	sqrt	平方根
	tan	正接関数，入力引数の単位は rad
複素数	imag	複素数の虚部

	complex	複素数の生成
	complex128	実部と虚部がそれぞれ64ビット浮動小数点へ変換
	real	複素数の実部
	unwrap	ジャンプのある位相特性の補正
ベクトル，配列	arange	等間隔の配列要素を生成
	array	配列の生成
	flipud	配列の上下反転
	hstack	配列を横に結合
	linspace	等間隔の数値の配列の生成
	meshgrid	二つの１次元配列から格子座標を生成
	ones	すべての要素が１の配列の生成
	shape	配列の各次元の大きさ
	size	配列の要素数
	sum	配列の要素の総和
	zeros	すべての要素が０の配列の生成
信号処理	fft.fft	高速フーリエ変換
	fft.fftshift	高速フーリエ変換の周波数0の成分を配列の中心に置くようにシフト
	fft.ifft	高速フーリエ逆変換
	random	乱数の生成

ii. 信号処理パッケージ signal

科学技術計算ライブラリ Scipy は数学的に高度な数値計算やアルゴリズムなどの大きなライブラリであり，その中の一つとして信号処理パッケージ signal がある．信号処理パッケージ signal には信号処理において必要とされるフィルタ，たたみこみ，フーリエ変換，線形システム解析などに関して膨大な関数がある．Python 対応 DSP では，signal の中の多数の関数が利用されている．以下のように scipy から signal を読み込んで，signal 中の関数を用いる．用いる際には signal.関数名 とする．

```
from scipy import signal
```

信号処理パッケージ signal の利用例を以下に示す．

```
# 信号処理パッケージ signal の利用の例
from scipy import signal

```

```
4:      win = signal.hamming(5)
5:      print('win =', win)
```

```
win =   [0.08 0.54 1.   0.54 0.08]
```

Python 対応 DSP で用いられている信号処理パッケージ signalの関数を以下にまとめている．

分類	関数名	説明
フィルタリング	convolve	たたみこみ
	convolve2d	たたみこみ（2次元データに対しての）
	lfilter	線形フィルタ，IIR フィルタ，FIR フィルタ
フーリエ変換	fft2	2 次元高速フーリエ変換，ただし from scipy.fftpack import fft2 によって読み込んで利用せよ
伝達関数と周波数応答	freqs	s 領域の伝達関数から周波数応答を計算
	freqz	z 領域の伝達関数から周波数応答を計算
	invresz	部分分数展開から z 領域の伝達関数を求める
	residue	s 領域の伝達関数から部分分数展開を求める
	tf2sos	伝達関数を2次セクションに変換
	zpk2tf	零点と極から伝達関数へ
FIR フィルタの設計	blackman	ブラックマン窓
	boxcar	矩形窓
	firwin	窓関数法によるFIR フィルタの設計
	hamming	ハミング窓
	hann	ハニング窓
	kaiser	カイザー窓
IIR フィルタの設計	bilinear	双線形変換によるディジタルフィルタの設計
	butter	設計仕様からバタワースフィルタの伝達関数を設計する
	buttord	設計仕様からバタワースフィルタの次数を求める

iii. 数学モジュール math

数学モジュール math は数学で用いられる多数の関数を提供している．numpy と重複する関数も多い．以下のように import 利用し，関数を利用する際にはmath.関数名 とする．

```
1:    # 数学モジュール math の利用
2:    import math
3:
4:    x = math.sqrt(2)
5:    print('x =', x)
```

```
1.4142135623730951
```

Python 対応 DSPにおいて用いている数学モジュール math の関数を以下にまとめている．

関数名	説明	備考
ceil	天井関数．引数以上の最小の整数	ceil(2.4) = 3 ceil(-2.4) = -2
floor	床関数．引数以下の最大の整数	floor(2.4) = 2 floor(-2.4) = -3

iv. Sys モジュール

これは Pythonインタープリタの定数，関数，メソッドに関しての情報を与えるモジュールである．import した後に sys.関数名 として利用する．

関数名	説明
exit()	プログラムを強制終了
float_info.epsilon	計算機イプシロン(2.22044e-16)．ゼロ割の回避や浮動小数点数（実数）の一致判定などに利用．

v. 自作関数

Python 対応 DSP はディジタル信号処理の基礎を学ぶためことを目的としているため，特に重要な信号処理手法については該当する関数を利用できるだけではなく，その実際的なアルゴリズムを理解することを重視している．また信号処理パッケージ signal では重要な関数が欠けている．これらの理由から Python 対応 DSP においてはいくつかの関数を自作している．以下の my のついた関数は自作の関数であり，my----.py の名前でファイルに保存されている．

自作関数を利用するためにあらかじめ以下のように関数を読み込んでおく必要がある．

```
from <モジュール名> import <オブジェクト名または関数名>
```

例えば mycircconv を利用する場合には

```
from mycircconv import mycircconv
```

として用いればよい.

Python 対応 DSP において用いている自作関数は以下に示すものである.

関数名	目的	対応する他の関数
mycircconv	循環たたみこみの直接計算	
myconvolve	たたみこみの直接計算	numpy.convolve , scipy.signal.convolve
mydft	離散フーリエ変換の直接計算	
mydirect	直接法によるディジタルフィルタの設計	
myfft	高速フーリエ変換	numpy.fft.fft , scipy.fft.fft
myfreqztrans	周波数変換によるディジタルフィルタの設計	
myimpinvar	インパルス不変法によるIIRフィルタの設計	
mylfilter	線形フィルタ，差分方程式, IIR フィルタ, FIR フィルタ	scipy.signal.lfilter

vi. Python の組み込み関数

第2章と第3章において Python の多数の組み込み関数組については説明がされているが，以下では Python 対応 DSP において用いられているその他の Python の組み込み関数をまとめている.

関数名	説明
abs	絶対値
format	値を他の書式に変換
len	ベクトルの要素数
max	ベクトルや行列の要素の最大値
min	ベクトルや行列の要素の最小値
print	印字
zip	複数のイテラブルオブジェクト（リストやタプル）の要素をまとめて取得

4. 参考資料

a. Python の入門

- C. Althoff (清水川監修) : 独学プログラマー Python,　日経BP, 2018.
- Guido van Rossum（鴨澤眞夫訳）: Python チュートリアル，オライリー・ジャパン，2007.
- 中久喜健司：科学技術計算のためのPython入門，技術評論社，2016.
- 林　真：「科学技術計算で使う」Python，工学社，2016.
- 国本大悟，須藤秋良：スッキリわかる Python 入門，インプレス，2019.
- Python チュートリアル（よく使われる機能を幅広く形式ばらずに紹介している入門サイト）
  https://docs.python.org/ja/3/tutorial/

b. Python の詳細

- Python 言語リファレンス（Python 言語の文法と重要なセマンティクスを正確さと完全さを優先して記述しているサイト）
  https://docs.python.org/ja/3/reference/index.html#the-python-language-reference
- Python コードのスタイルガイド（Python のコード作成において強く推奨されるプログラミングスタイルのガイド）
  https://pep8-ja.readthedocs.io/ja/latest/
- 拡張モジュール

数値計算モジュール Numpy

https://numpy.org/doc/stable/index.html#numpy-docs-mainpage

科学技術計算ライブラリ Scipy

https://scipy.org/scipylib/

数学関数モジュール math

https://docs.python.org/ja/3/library/math.html#module-math

グラフ描画ライブラリ Matplotlib の公式サイト

https://matplotlib.org/

Sys モジュール

https://docs.python.org/ja/3/library/sys.html

c. その他

- IPython の使い方メモ （マジックコマンド）

  https://www.magata.net/memo/index.php?IPython%A4%CE%BB%C8%A4%A4%CA%FD%A5%E1%A5%E2#d69e12d8

- Python の Matplotlib の使い方をまとめてみた

  https://qiita.com/renesisu727/items/24fc4cd8fa2635b00a0d

- 早く知っておきたかった Matplotlib の基礎知識

  https://qiita.com/skotaro/items/08dc0b8c5704c94eafb9

終り